What the frozen forests split on

A reading of the two random forests frozen on 12 October 2026. The trees were opened and counted. They were not retrained.

What we did

The detector has a network rule and two forests on the same 174 GPU features. The rule fires when a node's mean send+receive rate is at least 1 MiB/s and the two directions are balanced (the smaller is at least 0.8 of the larger). Stage 2 was fit only on windows where that rule fires: 3,019 training windows and 939 others, out of 3,958. Fallback was fit on every honest window: 10,059 windows, 244 jobs, 42 families. A window is called training when the forest that is allowed to speak has probability at least 0.5. Adversarial jobs were not in the fit.

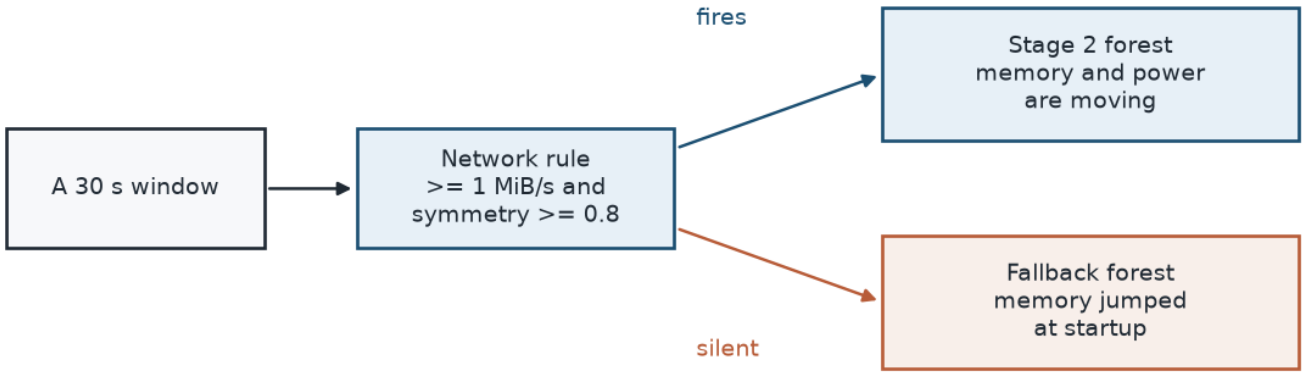


Figure 1. Which forest is asked. Once the rule fires, fallback is not consulted.

Each forest is 200 trees, minimum 2 samples in a leaf, 13 features drawn at each split, class weights balanced, seed 0. For every split we kept the feature, the threshold, and which side receives the training vote. How much a split counts, and why Figure 2 has five bars instead of 174, is spelled out under "Where the impurity goes".

This is not a permutation test. Shuffling a column on held-out jobs needs the training table, and that table is not what this note uses. The vote gap below is the tree's own class fraction on the high side of a cut, minus the fraction on the low side, averaged with the impurity weights. A gap near +1 means the high side votes training and the low side does not. Thresholds are impurity-weighted medians of many split points, not a single rule.

The two forests are not the same size

Stage 2	Fallback
Mean depth 4.8, deepest 9. About 10 leaves per tree. 162 of 174 features are split on at least once.	Mean depth 13.1, deepest 18. About 78 leaves per tree. 164 of 174 features are split on at least once.

Stage 2 is a short tree. Fallback is a real one. They also cut on different facts, which is the rest of this note.

Where the impurity goes

Each tree is a stack of yes/no questions. A question looks at one column ("is the spread of memory utilization above 1.2?") and sends the window left or right.

Before the question, training and not-training are mixed together. Gini measures that mix. With two classes it is 2p(1-p), where p is the share of the weight that is training. It equals 0.5 when the two classes are even, and 0 when every window on that side is the same class. A useful question is one that makes the mix drop. The drop at a node is

```
N * Gini - N_left * Gini_left - N_right * Gini_right
```

N is not a count of windows. The forest was fit with balanced class weights, so the scarcer class pulls harder than its share of the rows, and N is that weight, already stored on the node. We add the drop over every question in all 200 trees and give the sum to the column the question used. Divide by the total drop. That percentage is the bar. It is the same number sklearn calls feature importance. Recomputing it from the nodes matches sklearn to 0.0002.

The bars are not 174 separate columns. The spread, the range, and the interquartile range of memory utilization are one measurement saved three times. Drawn separately, one fact would look like three. Columns that come from the same place are added:

Bar	Which columns go into it
Memory	The name contains "mem": utilization, megabytes in use, and the jump in the first 30 samples.
Power and temperature	The name contains "power" or "temperature".
PCIe	The name starts with "pcie".
Compute	Whatever is left: GPU utilization, SM clock, idle fraction, tensor activity.
Video	Encoder and decoder. Those columns do not vary here, so the share is zero.

A column scores only at questions where the tree actually picked it. Each question is shown 13 of the 174 columns, so a twin that was not shown gets nothing for that question. And stage 2's memory bar of 61% does not mean memory caused the call. The startup jump and the utilization wobble are both inside that bar. Figure 2 is the total. Figures 3 and 4 take the bar apart.

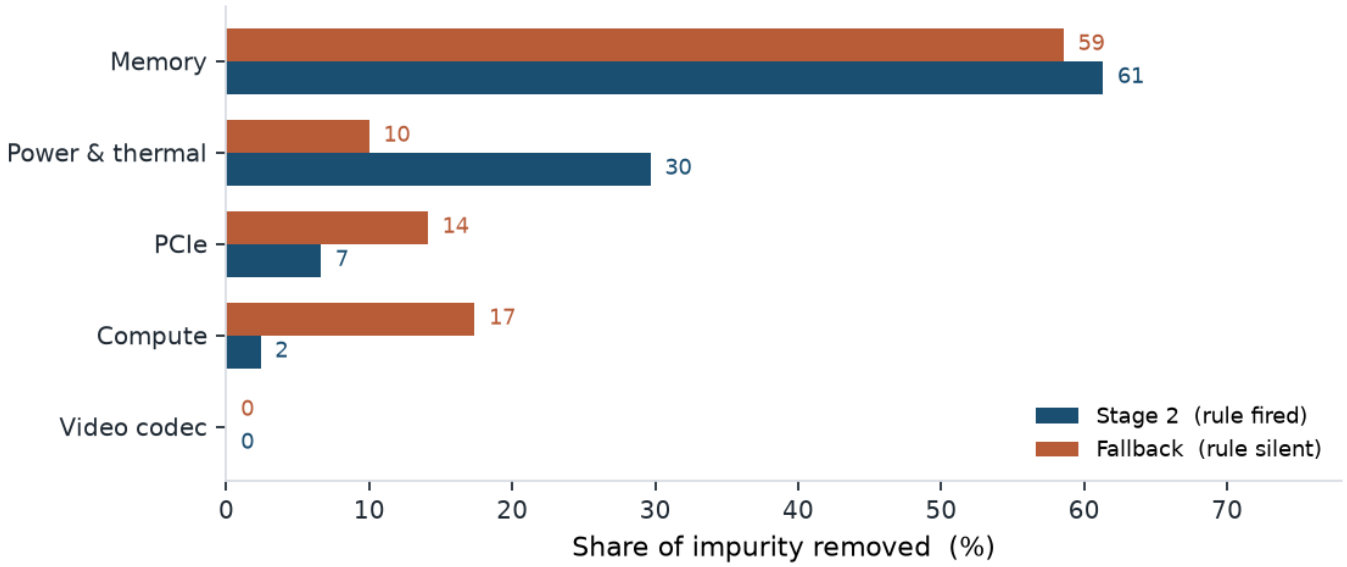


Figure 2. Of all the mix a forest removed, the share removed by questions about each source. Stage 2's memory bar is 61%: that share of the mix it removed came from a split on a column whose name contains "mem".

Stage 2 puts 61% on memory and 30% on power and temperature. PCIe is 7%. Compute, which is where GPU utilization and SM clock live, is 2%, and none of that is at the root. GPU-utilization mean is rank 105 and removes essentially no impurity.

Fallback is still 59% memory, but the memory it uses is the allocation at the start of the run. Compute rises to 17%, PCIe to 14%, and power falls to 10%.

Stage 2: a moving trace votes training

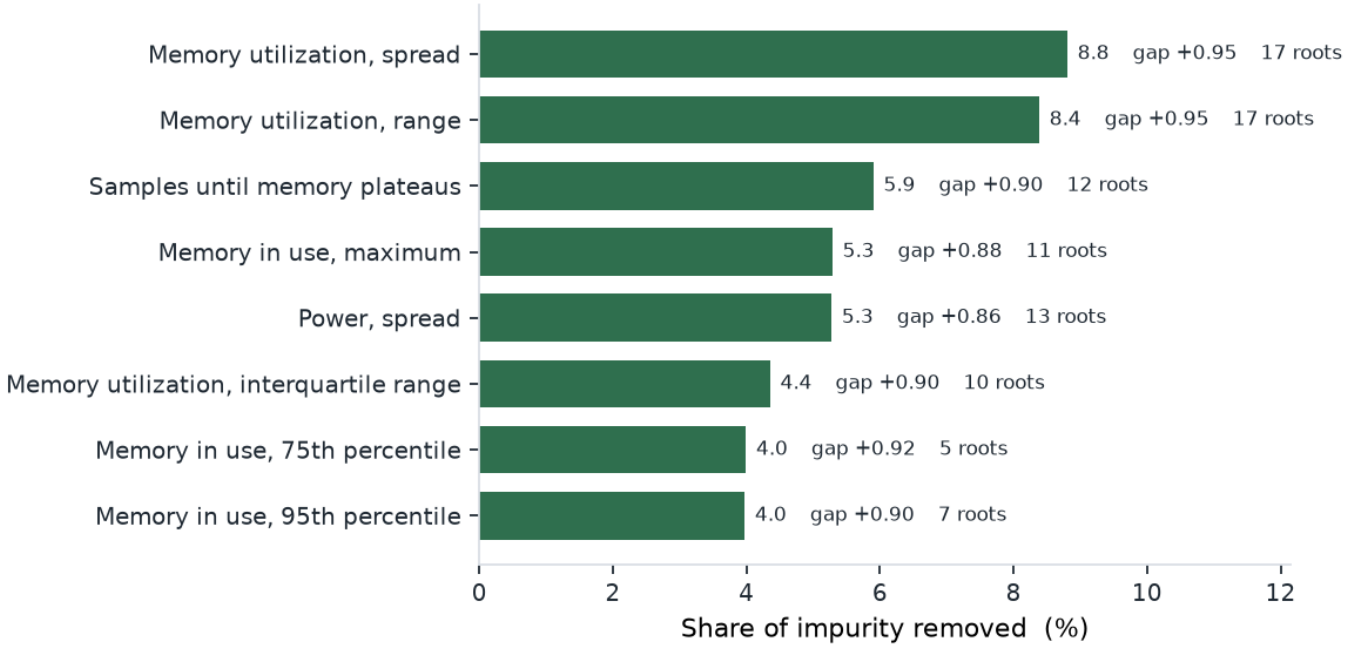


Figure 3. The eight heaviest features in stage 2. Green means the high side of the cut votes training. Several of these bars are one column: spread, range, and the interquartile range of memory utilization. Those three are the root of 44 of the 200 trees. Each split is shown only 13 of the 174 features, so a measurement that takes the root about as often as it is offered is one the tree actually prefers.

The cuts are monotone. Memory-utilization spread above about 1.2 percentage points (vote gap +0.95). Power spread above about 9 W (gap +0.86). Memory in use above about 10 GB (gap +0.88). Time to the memory plateau longer than about 16 samples (gap +0.90). The startup-allocation features that dominate fallback rank between 32 and 64 here.

The not-training windows in this fit are jobs with balanced traffic and a flat GPU trace. In the corpus those are HPC jobs, not inference. A serving window whose memory utilization moves is on the training side of every large cut. Stage 2 has no leaf whose job is to recognize inference.

Fallback: a startup allocation votes training

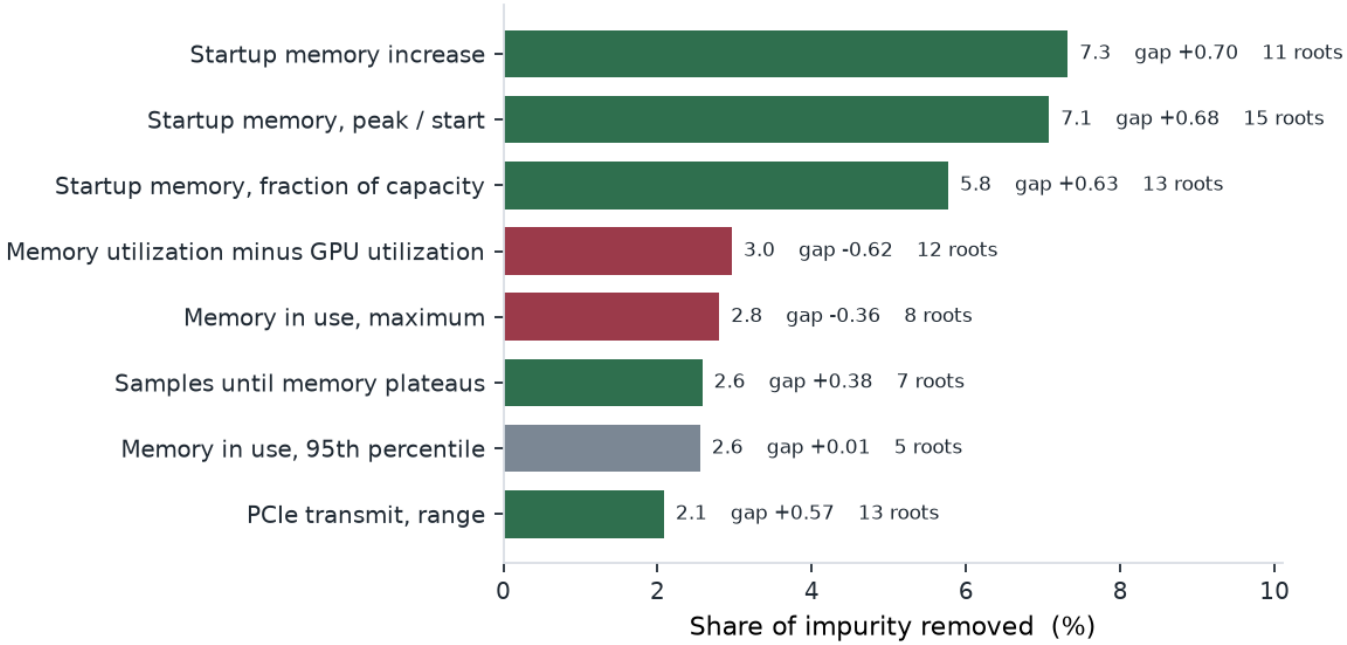


Figure 4. The eight heaviest features in fallback. Green: high side votes training. Red: high side votes not training. Gray: the two sides barely disagree. The first three bars are the same event. Together they are 20% of the impurity and 37% of the root weight.

In the feature code these columns read the first 30 samples of the run, as far as the current window is allowed to see. After those 30 samples, later windows of the same job all see the same jump. The comment on the function says it was written to catch optimizer state allocated at startup, on the order of 6 GB, which an inference process was not expected to do.

Typical cuts: an increase above about 4.9 GB (gap +0.70), a peak/start ratio above about 11 (gap +0.68), or an increase above 0.04 of capacity (gap +0.63).

The next cuts point the other way. Memory utilization minus GPU utilization has vote gap -0.62 and a typical cut near -23 percentage points: when the memory bus is busy relative to the SMs, the tree votes not training. Absolute footprint flips relative to stage 2. Fallback's typical cut on the maximum memory in use is 68 GB, and the high side votes not training (gap -0.36). Stage 2's cut on the same column is 10 GB, high side training.

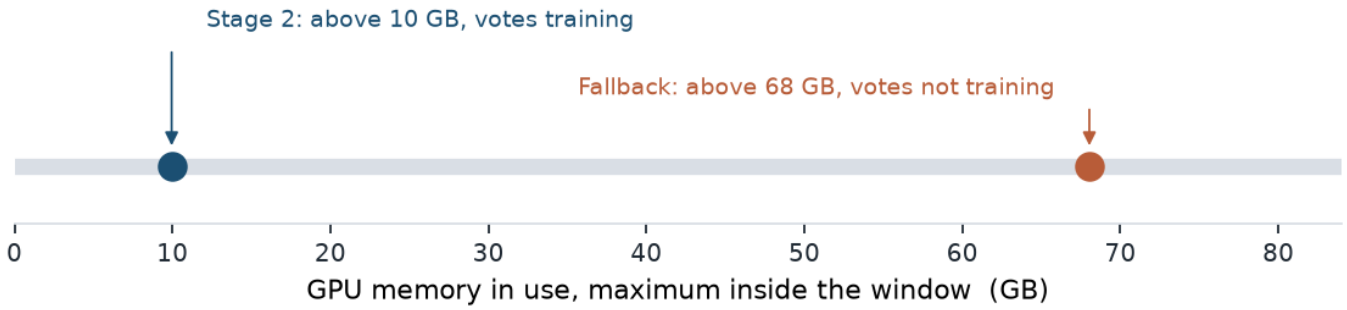


Figure 5. The same column, two forests, two directions. Stage 2 only ever saw balanced-traffic windows, where a footprint past about 10 GB separated training from flat HPC. Fallback also saw large serving jobs, and past about 68 GB it votes the other way. Both numbers are weighted medians of split points, not class medians.

GPU utilization is not the feature either forest leads with. In fallback its mean is rank 19, the typical cut is 99.7%, and the vote gap is only +0.52. A GPU has to be pinned before this column counts.

What that does to the gate

On the 93 Nebius jobs already scored with this frozen detector, the pattern matches the cuts. Those jobs were not used to choose features or to move the 0.5 cutoff. Probabilities below are the mean over the job's windows.

Workload	Rule	Stage 2	Fallback	Call
Training, IB off (iboff_flat)	silent	not asked	0.65	training
Training, no gradient collective (nosync_flat)	silent	not asked	0.62 to 0.66	training
LoRA, one node, IB off (lora_1n8_iboff)	silent	not asked	0.53 to 0.54	training on 85% of windows
Qwen serving plus a fake all-reduce (qwen_fake_ar)	fires	0.90 to 0.93	0.30 to 0.37	training
OLMoE across nodes (olmoe_ep_mixed)	fires	0.89 to 0.90	0.11 to 0.13	training
Split prefill/decode plus a return message (disagg_echo)	fires on most windows	0.91 to 0.95	0.35 to 0.47	training

Table 1. Orange rows are decided by fallback, because the rule is silent. Blue rows are decided by stage 2, because the rule fired. Fallback's probability on the blue rows is below 0.5 and is ignored.

The orange rows are training processes whose cross-node all-reduce is missing. Fallback sees the startup allocation and calls them training. That is the feature doing what it was built for. The blue rows are inference with balanced traffic on the wire. Stage 2 sees memory moving, the way it saw training, and calls them training. On every one of the 1,032 windows in this score where the rule fired, stage 2's probability was at least 0.62. It never vetoed.

Encoder utilization, decoder utilization, and the variation of the memory clock are never split on in either forest. Those columns are constant in the training corpus.

What this note is not

It does not say that memory spread causes a window to be training. It says that is the cut the saved trees use. Correlated copies of one signal share the impurity, which is why Figure 2 is the total to quote and Figures 3 and 4 are the detail. Nothing here refits a forest, adds the October campaigns, or retunes 0.5. The counting script is classifier_analysis/frozen_interp.py. Run it with scikit-learn 1.8 so the joblib files load as the same trees.